

EDSTEM 189 Portfolio

Gilberto Damian

CalTeach Program, UC Berkeley

EDSTEM 189: Integrating Research Methods into K-12

Teaching in Mathematics and Science

Dr. Morgan Mitchell

Summer 2026

Research

Research Paper

Toward Generalized Regulatory-Element Identification with Genomic Foundation Models

Gilberto Damian

Joint Genome Institute, Lawrence Berkeley National Laboratory - University of California, Berkeley

Mentors: Dr. Zhong Wang and Morgan Mitchell

Abstract

Locating promoters, ribosome-binding sites (RBS), and Rho-dependent terminators is foundational to prokaryotic genome annotation, yet the classical tools that do it are calibrated on a handful of model organisms and the dense genomic language models that might replace them treat every token with the same parameters. GenomeOcean-MoE, a sparsely upcycled 8-expert mixture-of-experts genomic foundation model, offers a third option: prior work showed its router encodes function in a genome-invariant way, distinct from the taxonomy-encoding hidden state. This work asks whether that routing channel identifies regulatory elements, and whether it does so better than classical annotation. Experimentally-derived ground truth was curated for all three elements (25 organisms for promoters, 6 for RBS, 3 for Rho), with each positive set validated against its own dinucleotide-preserving shuffle before modeling and probed via logistic regression on the frozen model under a six-class negative-control battery, reporting embedding, routing, and concatenated views separately. The central finding is a dissociation: for promoters and RBS the signal is carried almost entirely by the embedding and the router is effectively blind (routing MCC 0.017-0.246 for promoters, 0.040-0.359 for RBS, and 0.076 in the window-free per-token test), replicating across 8 bacterial phyla and 2 archaeal lineages. For Rho-dependent terminators the ordering inverts: routing exceeds embedding on the three controls that matter (D 0.498 vs 0.298, E 0.511 vs 0.233, F 0.968 vs 0.226), the only setting in this project where the MoE architecture contributes something a dense model could not. Scored head-to-head against five classical methods and two real published tools on identical held-out windows, GO-MoE ranks first of six on Rho, but second of seven on RBS behind real Prodigal (0.417 vs 0.470) and second of five on promoters behind a generic k-mer spectrum (0.265 vs 0.335). Three shuffle-controlled tests further show the Rho signal is compositional rather than arrangement-based, two independent functional checks (Term-seq 3'-end enrichment, bicyclomycin readthrough) confirm the underlying windows are real, and expert routing is therefore a genuine but narrow instrument: it helps precisely where a positioned consensus motif does not exist, and adds nothing where one does.

Introduction

Gene expression in bacteria is marked by short, structured DNA and RNA signals at the boundaries of the coding sequence. Promoters bring in RNA polymerase and set where transcription begins, recognized by the sigma-70 factor through conserved -10 and -35 hexamers. Ribosome-binding sites (RBS), typically the purine-rich Shine-Dalgarno motif complementary to the 3' end of 16S rRNA, position the ribosome for translation initiation (Shine & Dalgarno, 1975). Rho-dependent terminators end transcription when the Rho helicase loads onto a cytosine-rich, structure-poor stretch of nascent mRNA and dislodges the polymerase. Locating these three element classes matters for genome annotation, operon reconstruction, and synthetic circuit design.

Generalization is the main problem. An algorithmic identifier that works on *Escherichia coli* K-12 has not necessarily learned biology; it may have learned *E. coli*. Reference genome collections are phylogenetically and functionally skewed (Albright & Louca, 2023), and that skew transfers into any model trained or tuned on them, including sequence language models (Ding & Steinhardt, 2024). Generalized identification therefore means recovering these elements across taxonomically diverse, often uncured microbial genomes, exactly where existing methods are least validated.

That is why this work does not start with another classical tool or another dense genome model. Hand-engineered tools encode a fixed motif and carry forward whatever taxonomic bias is in the reference set used to build them. Dense genomic language models replace the fixed motif with a learned one, but the representation is still a single undifferentiated vector; nothing forces the model to separate "this looks like *E. coli*" from "this is a promoter." The expert-based GenomeOcean-MoE model routes each token to a sparse subset of experts instead, and prior work found that routing behaves as a genome-invariant, function-aligned channel distinct from the hidden-state embedding. If routing already separates function from genome identity, it is a plausible, previously untested substrate for identifying regulatory elements in a way that might survive the jump to an unseen genome.

This research asks whether that routing channel can identify and annotate promoters, RBS, and Rho-dependent terminators across diverse bacterial and archaeal genomes, including unseen, sequence-dissimilar ones, more accurately and generalizable than classical tools and dense genomic language models, and whether any advantage is causally attributable to expert specialization rather than incidental to the MoE architecture generally. The remainder of this paper lays out the experimental design, the ground-truth data, the falsification criteria, and the results.

Background

1. Classical algorithmic tools

Classical predictors record expert knowledge of a motif directly into a scoring function. For sigma-70 promoters, BPROM applies a linear discriminant function over -10/-35 position-weight matrices and oligonucleotide composition, reporting roughly 80% accuracy for the sigma-70 class, with parameters optimized on *E. coli* (Solovyev & Salamov, 2011). For RBS, the widely used gene caller Prodigal scores translation-initiation sites against a Shine-Dalgarno motif model, defaulting to the canonical SD sequence; roughly 88% of surveyed finished genomes use the SD motif (Hyatt et al., 2010). For Rho-dependent terminators, RhoTermPredict was the first exhaustive genome-wide search tool, using a two-step conserved-motif procedure calibrated on *E. coli*, *B. subtilis*, and *S. enterica* (Di Salvo et al., 2019).

Two limitations recur across all three. First, each is narrow: BPROM covers a single sigma class, Prodigal's RBS model struggles with non-canonical or leaderless starts, and RhoTermPredict's authors note that Rho's sequence determinants are "complex and poorly defined"; their genome-wide predictions matched RNA-seq evidence for only about 62% of candidate *E. coli* K-12 sites. Second, each is calibrated on only a handful of model organisms, mostly *E. coli* K-12. The reference-bias problem (Albright & Louca, 2023) in operational form: performance figures are reported on the same well-studied genomes the tools were tuned on.

2. Dense AI models and how GenomeOcean classifies

Genome foundation models replace hand-crafted motifs with representations learned by self-supervised pretraining on raw sequence. The history started with DNABERT, which established the transformer-for-DNA paradigm and the freeze-embed-and-probe evaluation style behind today's benchmarks (Ji et al., 2021), continuing through the Nucleotide Transformer family (Dalla-Torre et al., 2025), long-context HyenaDNA (Nguyen et al., 2023), and the generative Evo/Evo 2 models (Nguyen et al., 2024; Brixi et al., 2025). A parallel branch focuses on uncurated metagenomes, METAGENE-1 (Liu et al., 2025) and GenerAnno (Li et al., 2025), reinforcing that scale and data diversity improve representation quality (Hoffmann et al., 2022).

The dense GenomeOcean 4B model is the foundation of this project: a 4-billion-parameter generative genome foundation model trained on 645 Gbp of prokaryotic metagenomic co-assemblies (Hofmeyr et al., 2020), served through grouped-query attention (Ainslie et al., 2023) and FlashAttention-2 (Dao, 2023). Its motivation is to escape reference bias by learning from the microbiome as sampled rather than as curated (Wu, Zhou et al., 2025). It captures coarse phylogenetic structure and fine protein-coding constraints, corresponds linearly with Evo 2's embeddings, and generates DNA that folds into structurally plausible proteins by ESMFold pLDDT (Lin et al., 2023).

But how does GenomeOcean actually classify? Following the DNABERT-style protocol, hidden states are frozen, mean-pooled, and passed to a logistic-regression probe scored by MCC on the GUE benchmark (Hong, 2026). On the GUE promoter task the dense baselines reach 0.786 (GO-100M) and 0.799 (GO-500M) MCC, with GO-500M averaging 0.550 across the four tasks. Two caveats bear directly on the research question, however. First, GUE's tasks are eukaryotic and viral, cross-domain for a prokaryote-trained model, and its promoter task is human rather than sigma-dependent bacteria. Second, a dense model applies the same parameters to every token, with no built-in mechanism to specialize on distinct regulatory-element types. Dense foundation models thus provide strong general embeddings, yet their prokaryotic, regulatory-element-specific performance is still largely unmeasured.

3. The case for GenomeOcean-MoE

Mixture-of-Experts (MoE) architectures route each token to a small subset of specialized feed-forward "experts." In language modeling this gives dense-level quality at a fraction of the active parameters (Mixtral; Jiang et al., 2024) and can drive genuine functional specialization (DeepSeekMoE; Dai et al., 2024). Prokaryotic genomes are a natural fit: coding sequence, rRNA, tRNA, and intergenic DNA have distinct statistical signatures, so if routing captures that modularity it should buy both efficiency and biological interpretability.

Recent work by Hong (2026) is the first to apply MoE to a genomic language model, sparsely upcycling GenomeOcean into an 8-expert, top-2 architecture. Three findings make the extension to regulatory elements credible:

1. The upcycled model approaches the 4B dense parent's validation loss (5.497 vs. 5.437) with roughly 20x fewer active parameters.
2. Individual experts become enriched for specific content classes, a causally validated tRNA detector fires at ~13x its expected rate, and routing fingerprints alone predict functional class on held-out genomes with no annotation supervision (macro F1 = 0.665; tRNA F1 = 0.774).
3. The router encodes function (genome-invariant) while the continuous hidden state encodes taxonomy, and upcycling from the dense parent stabilizes routing under domain shift where a trained-from-scratch MoE silently collapses.

Together, these results show MoE routing captures biologically meaningful structure, but so far only for coarse content types (CDS, rRNA, tRNA, ncRNA, intergenic), evaluated largely on reference genomes. No study has yet asked whether that specialization extends to the finer, functionally pivotal regulatory elements, or whether a routing-aware GenomeOcean-MoE can identify them generalizably. That gap is the motivation for this work: classical tools are interpretable but narrow and E. coli-bound, dense foundation models generalize but treat all sequence uniformly, and sparse expert routing offers a mechanism to specialize on each element class while retaining metagenome-scale breadth.

Materials and Methods

The methodology has two parts, applied to each of the three regulatory element classes (sigma-70 promoters, ribosome-binding sites, and Rho-dependent terminators). First, ground-truth positive and negative sequence sets are curated from experimental data for a panel of bacterial and, where available, archaeal genomes, and independently validated before any modeling. Second, two linear-probe test batteries are run against the frozen GenomeOcean-MoE model's embedding and routing features: an Element Detection Test, testing decodability against a battery of negative controls designed to rule out trivial confounds, and a Per-Token Specialization Test, testing whether any decodable signal is specific to the element type rather than a generic regulatory marker. Both report embedding, routing, and concatenated views, since whether routing specifically carries the signal is the central question motivating the MoE architecture. A third component benchmarks named classical annotation tools on the identical held-out windows so the comparison is direct rather than cited.

Ground-Truth Data and Genome Panels

Every positive and negative label is drawn from experimental evidence: dRNA-seq or TSS-mapping for promoters, ribosome profiling for RBS, and Term-seq, bicyclomycin (BCM) readthrough, or genetic Rho-depletion for Rho-dependent terminators, never from an algorithmic predictor's output. This anti-circularity rule ensures the classical tools evaluated here (BPROM, Prodigal, RhoTermPredict) are always scored against independent experimental truth. Sources are tiered by evidence strength: genome-wide evidence from living organisms (Tier 1) is the headline positive class, lower-resolution supplementary evidence (Tier 2) is tracked separately, and algorithm-derived candidates are retained only as a cross-check, never as ground truth.

Element	Panel	Sources
sigma-70 / archaeal promoters	25 organisms, 8 bacterial phyla + 2 archaeal lineages	Prokaryotic Promoter Database (PPD), 129,148 experimentally verified promoters across 63 species (Su et al., 2021); archaeal TATA/BRE consensus sourced from published <i>H. volcanii</i> / <i>H. salinarum</i> dRNA-seq studies
Ribosome-binding sites (RBS)	6 organisms, 5 lineages (Gammaproteobacteria, Firmicutes x2, Actinobacteria, Alphaproteobacteria, Archaea)	Ribosome profiling / Ribo-RET translation-initiation-site maps: <i>E. coli</i> K-12 (Meydan et al., 2019) and TetRP (Nakahigashi et al., 2016); <i>M. tuberculosis</i> (Sawyer et al., 2021); <i>B. subtilis</i> (Lalanne et al., 2018); <i>C. crescentus</i> (Schrader et al., 2014); <i>H. volcanii</i> (Gelsinger et al., 2020)
Rho-dependent terminators	3 organisms, 3 phyla (Gammaproteobacteria, Actinobacteria, Spirochaetes)	<i>E. coli</i> : BCM readthrough + Term-seq concordance (Peters et al., 2012; GEO GSE109766); <i>M. tuberculosis</i> : Term-seq + genetic RhoDUC depletion (D'Halluin et al., 2023); <i>B. burgdorferi</i> : $\pm$ BCM RNA-seq readthrough (Petroni et al., 2023). Intrinsic (Rho-independent) terminators, TERMITE atlas (<i>E. coli</i> , <i>B. subtilis</i>)

Table 1: For elements with sufficient positive-class volume (promoters, RBS, and the Rho in vivo panel), sequences are clustered at $\leq 60\%$ identity (MMseqs2 easy-cluster) and split 80/20 at the cluster level, so held-out evaluation is against sequence-dissimilar examples rather than near-duplicates of the training set. Supplementary, lower-tier evidence receives a simple per-organism random 80/20 split instead, since it is never used to support the sequence-dissimilarity claim.

Positive-Control Validation

Before any probe is run, every positive-control set is checked against its own dinucleotide-preserving shuffle (Altschul-Erikson algorithm) to confirm the windows contain genuine biological signal rather than an artifact of the extraction pipeline. For promoters and RBS this is a motif-enrichment check: the domain-appropriate consensus element (sigma-70 -10/-35 hexamers, archaeal TATA/BRE, or the Shine-Dalgarno core) is searched in the real window and its shuffled twin, with significance assessed by exact McNemar test. Because shuffling preserves base composition by construction, it cannot validate Rho-dependent terminators, whose defining feature is composition itself; Rho windows are instead checked for base-composition enrichment against a matched background class. Any organism whose positive windows fail this check is dropped from the detection test for that element.

Frozen-Model Feature Extraction

GenomeOcean-MoE is used strictly as a frozen feature extractor: it is never fine-tuned, and no gradient flows into its weights. For each input sequence, two feature views are recorded from a single forward pass: the embedding view (the mean-pooled final-layer hidden state, 768 dimensions) and the routing view (the router's expert-selection logits mean-pooled across all layers and experts, 96 dimensions: 12 layers x 8 experts). A logistic-regression probe is then trained on top of these frozen features for each test. Results are described as probes rather than classifiers because a probe measures what is already linearly decodable from a fixed representation, not what a trainable system could be taught to extract.

Sequence enters the model through a byte-pair-encoding tokenizer with a 4,096-token vocabulary trained directly on DNA, so each token is a variable-length nucleotide substring rather than a fixed unit such as a base or codon. Averaged across the vocabulary, one token spans approximately 4.89 bp: a 50 bp window is roughly 10 tokens, and a short consensus motif (6 to 8 bp) occupies only 1 to 2 of them, with the exact boundary depending on the surrounding sequence.

Test 1: Element Detection Test

The Detection Test asks the most basic question for each element: is it linearly decodable from the frozen model's features, and does that decodability survive a battery of negative controls designed to strip out the confounds most likely to inflate an apparent signal (composition, positional registration, neighboring-gene content)? Each element uses a single fixed-size window anchored on its own experimentally-defined landmark.

Test 1: Window Construction

The anchor convention differs by whether the landmark is a trustworthy single base pair. A transcription start site is trustworthy at single-bp resolution, so the promoter window is 50 bp (offsets [-49, 0] relative to the TSS), with the TSS as the rightmost base. A called Rho termination site (TTS), by contrast, is the noisy summary point of a much broader termination zone (a single called site sat inside a >1,200 bp effective range against the underlying Term-seq pileup) and is treated as unreliable at single-bp resolution: the Rho window is 200 bp (offsets [-200, -1]), with the TTS itself excluded. The RBS window follows the same excluded-anchor convention, 50 bp at offsets [-50, -1] relative to the start codon, so neither the start codon nor coding sequence enters the window.

Test 1: Negative Control Battery

One binary probe is run per negative class, always positive-vs-negative, never pooled across classes. Table 2 lists the six controls; not every element uses all six.

Class	Construction	Tests
A	Dinucleotide-preserving shuffle of the positive window	Composition/order. Kills positional motif structure while holding GC and dinucleotide statistics fixed
B	Shift-decoy, near offset (20-150 bp depending on element)	Positional registration. Same locus, landmark no longer at the anchor
C	Shift-decoy, far offset (80-600 bp depending on element)	Same genomic neighborhood, element out of frame. Isolates neighboring-gene content from the element signal itself
D	Generic intergenic sequence, matched distance from any known site	Non-coding background, unrelated to any regulatory element
E	Sequence drawn from the interior of a coding sequence	The coding-vs-noncoding axis. Expected to be the easiest contrast, not the specificity headline
F (Rho only)	Same-position window upstream of an intrinsic (Rho-independent) terminator	Termination-mechanism specificity (the one test not available for promoters or RBS.)

Table 2: *Negative-control battery for the Element Detection Test.*

Test 1: Probe Protocol

For every (positive, negative) pair, examples are sampled in matched proportion per organism, capped at the smaller of the positive and negative counts available, so a probe cannot succeed merely by learning genome identity. A logistic-regression classifier (L2-regularized, balanced class weights) is fitted on standardized features from the training split and evaluated on the held-out split, separately for each feature view. The primary metric is the Matthews correlation coefficient (MCC); false-positive and false-negative rates are reported alongside it.

Test 2: Per-Token Specialization Test

The Detection Test establishes whether an element is decodable at all; it cannot establish whether that decodability is specific to the element's own identity or reflects a generic regulatory-vs-background axis. The Per-Token Specialization Test answers that second question directly by removing window construction entirely, eliminating every window-boundary confound the Detection Test's negative controls exist to guard against.

Test 2: Genome-Coordinate Labeling

Rather than extracting a fixed-size window per instance, every experimentally confirmed element position is recorded as an exact genome-coordinate span (e.g., the -35 hexamer's own 22 bp span), directly on the genome, on both strands. Two matched negative constructions are generated per label: a random intergenic span (BG) and a random coding-sequence span (CDS), drawn from the same organism in comparable number. A third comparison, XELEM, compares any two labeled element types sharing at least one organism directly against each other, the cleanest available test of whether the representation distinguishes element identities rather than a shared regulatory-or-not signal.

Test 2: Whole-Genome Chunked Extraction

Each genome is tokenized in overlapping ~4,800 bp chunks (50% overlap) on both strands, and the frozen model is run once per chunk; every labeled position inside a chunk reuses that chunk's own embedding and routing output rather than requiring a separate forward pass. When a label falls inside more than one overlapping chunk, only the single most-centered copy is kept, never averaged, since pooling embeddings from two different contexts has no clean interpretation.

Test 2: Probe Protocol

Probes follow the same convention as the Detection Test: organism-matched sampling, logistic regression on standardized features, embedding / routing / concat views, MCC as the primary metric, applied instead to the BG, CDS, and XELEM comparisons across all three element classes.

Test 3: Classical-Tool Benchmark

Citing a classical tool's published accuracy figure and comparing it to a number produced here would not be a fair comparison, since the two come from different data under different metrics. The third test instead scores classical methods directly on the same held-out windows, with the same sampling, seeds, and MCC metric. Two categories of methods are run. The first category is reimplemented mechanisms: the underlying scoring principle of a published tool, coded against window sequences. These are mononucleotide-plus-GC composition, dinucleotide composition, a consensus-motif position-weight-matrix scan (the BPROM and RBSfinder mechanism), Shine-Dalgarno hybridization free energy computed with ViennaRNA duplexfold against the organism's own anti-SD 16S tail (the RBS Calculator and Free2Bind mechanism; Lorenz et al., 2011), and a RhoTermPredict-style C-greater-than-G scan.

The second category is real published tool code, so the comparison cannot be dismissed as a strawman. Real RhoTermPredict was cloned from its authors' repository; because the published script is an interactive genome scanner with no importable interface, its rut-site regular expression, refinement step, and hairpin/pause logic were extracted verbatim and rehosted as pure functions. Real Prodigal was run through the pyrodigal binding (Larralde, 2022) in metagenomic mode; since Prodigal only scores an RBS as part of full gene calling, a synthetic downstream lacZ fragment was appended to each window to force a gene call from which the RBS sub-score could be extracted.

The harness was validated before its numbers were trusted. Composition and dinucleotide methods score exactly 0.000 against negative control A, correct and expected since the dinucleotide-preserving shuffle holds those statistics fixed by construction. The motif-PWM method still detects the element under the same control, also correct, since shuffling destroys the positional motif while leaving composition untouched.

Robustness and Validation Procedures

Four additional procedures support the results below. First, because the Rho panel contains only three organisms, point MCCs for that element are accompanied by a genome-level bootstrap (organisms, not windows, resampled with replacement over 2,000 replicates), reflecting uncertainty about generalizing to a new genome; Control F, present only in *E. coli*, admits only a site-level bootstrap capturing within-organism noise.

Second, a window-shift robustness test displaces the window from its anchor in graded bands and re-runs the probe at each offset, distinguishing a landmark-registered signal from one smeared across the neighborhood. Third, a composition-orthogonal battery asks whether the Rho signal survives holding composition fixed, comparing each real window against 1,000 of its own shuffles under three candidate features: secondary-structure free energy, cytosine periodicity, and CCC-tract clustering.

Fourth, and independent of sequence composition entirely, the *E. coli* Rho windows are checked against two functional readouts: the raw Term-seq 3'-end pileup, testing whether each called site is a genuine local termination peak, and the bicyclomycin readthrough measurement from Peters et al. (2012), the assay that operationally defines Rho-dependence. These checks address label validity rather than model performance and are reported separately for that reason.

Statistical Conventions

Three conventions apply across both tests. First, organism-matched sampling is used in every probe, so no result can be explained by a probe learning genome identity rather than the element itself. Second, every number is produced by a frozen, never-fine-tuned model with a logistic-regression probe on top, hence the term "probe" rather than "classifier," following interpretability-literature convention. Third, results are always reported as three parallel numbers (embedding, routing, concatenation) rather than a single aggregate, since whether the routing channel specifically carries the signal is the throughline empirical question this project is designed to answer.

Falsification Criteria

Two claims are held to a falsifiable standard. The detection claim fails if the routing-aware detector does not beat the named classical tools and dense language models on held-out genomes, or if its advantage collapses toward the GC-content-matched chance baseline on the sequence-dissimilar, unseen split. The causal claim fails separately: if masking the expert(s) responsible for a given element produces no meaningfully larger increase in error than masking a matched control expert, or if a version of GenomeOcean-MoE without specialized experts reproduces the same detection result, the advantage is incidental to the architecture rather than carried by expert specialization.

Results

Window Validity Precedes Every Reported Number

No detection result is reported for a positive set that did not first clear its own validity check. For promoters, consensus-motif enrichment against the shuffle replicated across the full 25-organism panel spanning 8 bacterial phyla and 2 archaeal lineages (pooled exact-McNemar p-values from 2.5×10^{-12} to effectively zero, with two genuine exceptions flagged and excluded). The RBS panel cleared the equivalent Shine-Dalgarno check. This ordering matters: when a probe later reads near zero, the windows have already been shown to contain the expected biology, so a weak result is a statement about the representation rather than about the data.

Test 1: Element Detection

Figure 1 reports held-out MCC for each element under each negative control, in all three feature views. The result is a dissociation, and it is the central empirical finding of this work.

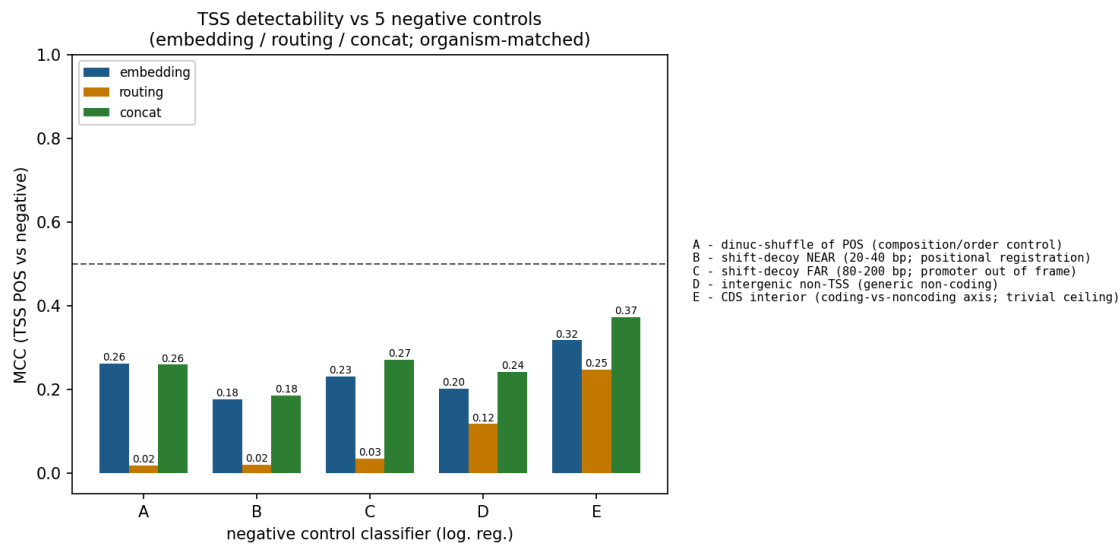


Figure 1a: Promoter held-out detection MCC by negative control and feature view.

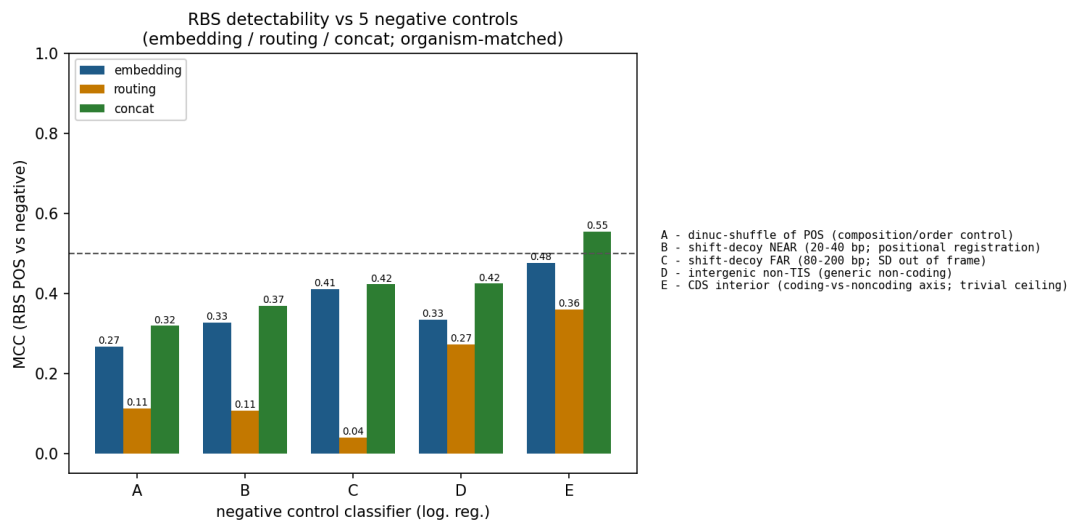


Figure 1b: RBS held-out detection MCC by negative control and feature view.

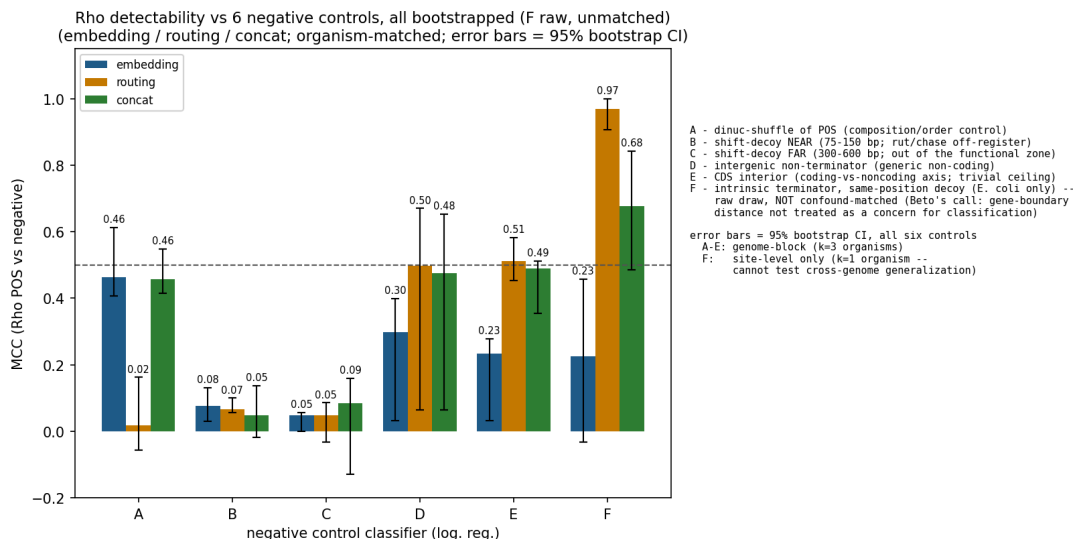


Figure 1c: Rho-dependent terminator held-out detection MCC by negative control and feature view, with 95% bootstrap confidence intervals. Control F is available only for Rho-dependent terminators and only in *E. coli*, and is shown unmatched for the gene-boundary-distance confound (see below).

For promoters and RBS, the embedding carries essentially all of the decodable signal and the router is close to blind. Promoter routing ranges from 0.017 to 0.246 and RBS routing from 0.040 to 0.359, with the upper end occurring on control E, the coding-versus-noncoding contrast, the easiest and least specificity-informative comparison in the battery. Concatenating routing onto the embedding produces only marginal movement over the embedding alone. This pattern held across every organism in both the promoter and RBS panels.

For Rho-dependent terminators the ordering inverts. Routing exceeds embedding on control D (0.498 versus 0.298), control E (0.511 versus 0.233), and most sharply control F (0.968 versus 0.226), the same-position comparison against intrinsic, Rho-independent terminators. Control F is the cleanest case for the MoE architecture here, since positive and negative windows occupy the same structural position and differ only in termination mechanism; it is also the only place in this work where the routing channel supplies something a dense model has no mechanism to supply.

Two caveats attach to the Rho numbers. Controls B and C sit near chance for all views (0.049 to 0.085), meaning the signal is not registered to the termination site at single-base resolution, consistent with the noisy-anchor rationale for excluding the TTS, and with only three organisms the genome-level bootstrap on control D spans [0.065, 0.670] at 95% confidence (control E is tighter, at [0.454, 0.583]). Control F also carries a partially resolved confound: positive windows sit farther from the upstream gene's end (median 537 bp) than intrinsic-terminator windows (median 338 bp), and matching positives and negatives one-to-one on that distance lowers routing from 0.968 to 0.842 (site-level bootstrap [0.714, 0.969].) The more conservative number to prefer for any claim about mechanism specificity.

Window-Shift Robustness

Displacing the window from its anchor separates a landmark-registered signal from a diffuse neighborhood signal, and the two elements tested behave differently. RBS decays smoothly from a base MCC of 0.554 through 0.526, 0.478, 0.471, 0.472, and 0.436 across shift bands from 5 to 50 bp, the profile expected when a real motif sits at a trustworthy anchor and is progressively shifted out of frame. Rho instead drops sharply at the first displacement (0.489 to 0.302) and plateaus around 0.24-0.30 across 200 bp of further shift, consistent with the anchor summarizing a broad termination zone rather than marking a discrete functional point.

Test 2: Per-Token Specialization

Removing window construction entirely strengthens rather than changes the conclusion. In the coordinate-exact per-token test, the Shine-Dalgarno label produced the strongest embedding signal recorded anywhere in this project (0.507 against intergenic background, 0.666 against coding sequence), while routing on the identical labels remained at 0.076 and 0.077; the -10 hexamer and archaeal TATA box show the same shape (0.358 and 0.577 embedding versus 0.091 and 0.110 routing). The XELEM comparison, which pits two element types directly against each other and is the sharpest available test of specialization, gives embedding MCCs of 0.39 to 0.51 and routing MCCs of -0.022 to 0.042: the router fails not only to separate elements from background but to separate one element type from another, the specific capability the mixture-of-experts architecture was hypothesized to provide.

Test 3: GO-MoE Versus Classical Tools

Figure 2 gives the head-to-head comparison, all methods scored on identical held-out windows under identical sampling and metric, reported as mean MCC across negative controls A through E.

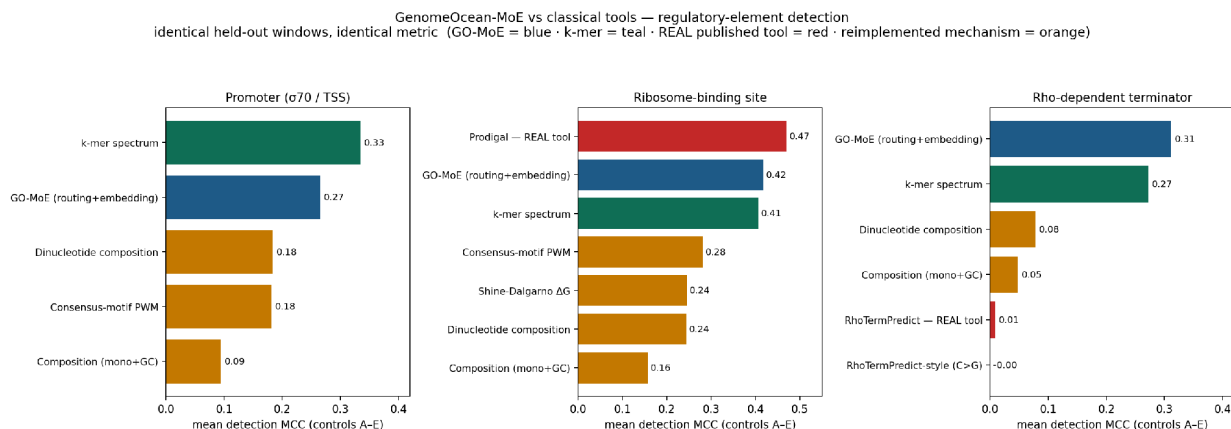


Figure 2: Mean detection MCC over negative controls A-E, by element and method. All methods scored on the identical held-out windows with organism-matched sampling and identical random seeds. Blue = GO-MoE (this work); teal = k-mer spectrum baseline; red = real published tool; orange = reimplemented classical mechanism.

GO-MoE beats every reimplemented classical mechanism on all three elements, which is the flattering reading and also the least informative one, since a reimplementation on a 50 bp window is not the tool its authors shipped. The informative comparisons are the other two: on RBS, real Prodigal edges GO-MoE out, 0.470 to 0.417, as a purpose-built gene caller from 2010 running against a 4-billion-parameter foundation model; on promoters, a generic k-mer spectrum beats GO-MoE outright, 0.335 to 0.265, meaning the learned representation does not exceed what simple sequence statistics already capture for that element.

GO-MoE ranks first only on Rho-dependent terminators, at 0.311 against 0.272 for the k-mer baseline, and real RhoTermPredict scores 0.008, indistinguishable from chance. That figure requires an honest qualification rather than a victory lap: RhoTermPredict needs roughly 150 bp of downstream context to evaluate hairpin and pause structure, and the anchor-excluded window design leaves only about 29 bp of it available. The defensible statement is that GO-MoE wins where the classical tool's own design assumptions are violated by a clean-window setup, not that it is a better Rho terminator predictor in general deployment.

Is the Rho Signal Compositional?

Rho performed conspicuously better than promoters and RBS on several controls, which warranted skepticism rather than acceptance. Holding dinucleotide composition fixed and comparing each real window against 1,000 of its own shuffles produced a clean null under every candidate feature tested. Secondary structure returned significance for 2.1% of 1,118 windows against a 5% chance expectation (binomial $p \approx 1.0$), and the p-value distribution skewed toward 1.0, meaning real windows were slightly more structured than their shuffles, the opposite of the unstructured-rut prediction. Cytosine periodicity returned 4.4% (binomial $p = 0.85$). CCC-tract clustering returned 1.16% pooled. The pattern held per organism.

The C-versus-G bubble test against matched background gave a partial exception: *B. burgdorferi* passes clearly (+1.56, $p = 0.0004$), *E. coli* reads null ($p = 0.87$), and *M. tuberculosis* trends in the wrong direction ($p = 0.998$), likely a metric limitation given its already GC-rich genome rather than a biological finding.

Two attempted metrics, a 40 bp cytosine-periodicity autocorrelation and an RNAfold-based structure test, both read null even for *B. burgdorferi*, the one organism independently shown to carry real composition signal; a metric that cannot detect a known signal cannot serve as evidence that other signals are absent, so these are reported as failed tests rather than null findings.

The conclusion, stated carefully, is that every composition-orthogonal feature tested comes back null once composition is held fixed. Whatever makes a Rho window detectable is compositional rather than a positioned motif or a spatial arrangement, matching the literature's description of rut sites as degenerate, cytosine-rich, guanine-poor regions with no consensus sequence, unlike promoters and RBS which have real positional motifs.

Independent Validation That the Rho Windows Are Real

A null result about arrangement invites the objection that the labels themselves are wrong. Two checks that do not depend on sequence composition address this for *E. coli*, the one organism with both a locally available raw pileup and a published readthrough table. Against the raw per-position Term-seq 3'-end read counts, all 151 *E. coli* sites are unambiguous local spikes, with a mean fold-enrichment of 3,220x over local background; every site falls in the top 5% of its own ± 200 bp neighborhood and 92.7% exceed every other position nearby. Against the bicyclomycin readthrough measurement, which is the assay that defines Rho-dependence, all 151 sites show a median tenfold increase in downstream transcription when Rho is chemically inhibited (99.3% positive, 84.1% exceeding fourfold), sitting at the stronger end of the full 1,264-site Peters population (median 3.34 versus 2.93) rather than at the marginal end.

One honesty note belongs with this result. These sites were partly selected for being BCM-significant in the first place, so responding to BCM at all is partly definitional; the magnitude of the response, and the Term-seq peak sharpness, are the independent parts of the claim. A second scope limit is that both checks are *E. coli* only; validity for *M. tuberculosis* and *B. burgdorferi* still rests on their source publications' own significance filtering. Taken together, the null arrangement result and the positive functional result resolve cleanly rather than conflicting: the windows are functionally confirmed Rho terminators whose composition-independent sequence features simply do not carry additional signal, a negative result about which features are detectable, not a validity problem with the labels.

Conclusions

The falsification criteria set out in the methodology give a mixed verdict under honest application. The detection claim, as stated, fails: a routing-aware detector was required to beat the named classical tools on held-out genomes, and it does not, since real Prodigal edges it out on RBS and a k-mer spectrum beats it on

promoters. The claim survives only on Rho-dependent terminators, partly because the comparison tool cannot operate under the window conditions imposed; reporting this as a general win for foundation models over classical annotation would misrepresent what the numbers show.

What does hold, and holds strongly, is a structural finding that was not the original hypothesis: the router is not a general-purpose regulatory-element channel. Across promoters and RBS, across 8 bacterial phyla and 2 archaeal lineages, in both windowed and window-free designs, routing MCC stays near zero while the embedding carries real signal, and the XELEM comparison shows the router cannot even distinguish one element type from another. Prior work established that routing predicts coarse content class (CDS versus rRNA versus tRNA versus intergenic) in a genome-invariant way; the present results locate the boundary of that capability, which does not extend downward to fine-grained regulatory elements defined by positional motifs.

The Rho exception sharpens rather than contradicts this. Rho is the one element with no consensus motif, defined instead by degenerate compositional character across an extended region, and it is the one element where routing outperforms the embedding. A coherent reading is that the router encodes broad compositional and content-type character rather than positioned sequence features: where an element is defined by that kind of character, routing helps; where an element is defined by a motif at a position, routing has nothing to contribute and the embedding does the work. Control F is the strongest evidence for this reading, separating the two terminator types at 0.968 unmatched (0.842 with the distance confound controlled) while the embedding manages only 0.226.

This has a practical consequence for anyone deciding whether a mixture-of-experts architecture is worth its complexity for genome annotation: the answer is conditional on element type. For motif-defined elements, a dense model gives the same signal and the routing channel is dead weight. For compositionally-defined elements with no consensus sequence, which are precisely the elements classical tools handle worst, routing supplies a discriminative signal a dense model has no architectural mechanism to expose.

The strength of the k-mer baseline deserves emphasis. A simple k-mer spectrum beat a 4-billion-parameter foundation model on promoter detection and came within 0.01 MCC of it on RBS. Combined with the finding that the Rho signal disappears once composition is held fixed, regulatory-element detectability in this setting appears largely compositional: foundation-model representations are not adding as much over sequence statistics as their parameter count would suggest, and future claims in this area should clear a k-mer baseline before being taken seriously.

Limitations and Sources of Variability

The Rho panel is the most serious constraint. Three organisms is enough to demonstrate an effect but not to characterize its variability: the genome-level bootstrap makes that concrete, with control D's 95% interval spanning [0.065, 0.670]. Control F, the strongest result in this work, exists in only one organism and supports no cross-genome generalization claim; its site-level interval describes within-*E. coli* noise and nothing more.

Tokenization granularity is a limitation that applies to every element, not just Rho. Because the BPE tokenizer's segmentation averages 4.89 bp per token and is not aligned to biological boundaries, a 6 to 8 bp consensus motif can fall entirely inside a single token, be split across two, or shift which token contains it depending on the surrounding sequence. This is a plausible partial explanation for why routing reads near zero for promoters and RBS even though the embedding detects the same motif: a signal confined to one or two tokens is diluted by mean-pooling before the probe ever sees it. The per-token test (Test 2) reproduces the same near-zero result at single-token resolution, arguing against pooling dilution as the full explanation, though a subtler boundary-straddling version of the same problem is not directly ruled out.

The classical-tool comparison is constrained by the window design in a way that cuts against the classical tools: both real tools were built to operate on genomes with gene context available, and the anchor-excluded window deliberately withholds it. For RhoTermPredict this is disqualifying rather than merely disadvantageous, since roughly 29 of its required 150 bp of downstream context survives. A fair reading treats the reimplemented-mechanism column as a lower bound on native-deployment performance.

The OSTIR implementation of the RBS Calculator was wired up but its precompute did not finish in time and is not included in any reported number; it is the most likely candidate to change the RBS ranking, being a thermodynamically principled method that Prodigal's simpler SD scoring does not subsume.

Test 2 has been completed for promoters and RBS but not for Rho-dependent terminators; the coordinate-exact labels are built and extraction is pending. The strongest claim in this work, the Rho routing advantage, has therefore been established only in the windowed design, and confirming it under the window-free design (which eliminates boundary confounds) is the single most valuable outstanding piece of evidence.

Finally, the causal claim has not been tested. Everything reported here is detection, whether a signal is linearly decodable from frozen features, and whether the Rho advantage is carried by specialized experts requires expert-ablation experiments that have not been run. Until they are, routing carries Rho signal; expert specialization causing it remains unconfirmed.

Summary

In summary, the expert-routing channel of GenomeOcean-MoE fails as a general regulatory-element detector but succeeds in one specific, well-characterized place. For motif-defined elements (promoters, RBS) the router is effectively blind and the embedding carries essentially all decodable signal; for Rho-dependent terminators, which have no consensus motif, routing exceeds the embedding on every control that matters. Against classical tools, GO-MoE ranks first of six on Rho but second of seven on RBS behind real Prodigal and second of five on promoters behind a k-mer spectrum. Expert routing is therefore a narrow instrument rather than a general one: it contributes exactly where a positioned consensus motif does not exist, and nothing where one does.

Acknowledgements

Zhong Wang and Morgan Mitchell for their mentorship and guidance throughout this project. Arash Jamshidi for running and facilitating the Summer Research Institute. Elisa Stone for the guidelines and advice on writing this research paper. Junho Wong of Northwestern University, who began the work that this project continued. Nicholas Pratt and LeAnn Lindsey of the Joint Genome Institute, and Lizhen Shi of Northwestern University, for their expertise and support. Anthony Wang of UC Merced and the Joint Genome Institute for his collaboration along the way. This work was prepared in fulfillment of the requirements of the Summer Research Institute, managed by CalTeach at UC Berkeley (EDSTEM 189), with support from UC Merced, the Joint Genome Institute, and the United States Department of Energy. Computational work was performed on the Lawrence Berkeley National Laboratory Research Computing cluster.

Data and Code Availability

All ground-truth data derives from previously published experimental datasets cited in Table 1 and in the references. Analysis pipelines, negative-control construction, probe harnesses, the classical-tool benchmark, and all result files and figures are maintained in the project repository. Every reported MCC is reproducible from the committed result files and the seeds recorded in the corresponding pipeline scripts.

References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv:2305.13245*.
- Albright, S., & Louca, S. (2023). Trait biases in microbial reference genomes. *Scientific Data*, 10(1), 84.
- Altschul, S. F., & Erikson, B. W. (1985). Significance of nucleotide sequence alignments: a method for random sequence permutation that preserves dinucleotide and codon usage. *Molecular Biology and Evolution*, 2(6), 526-538.
- Brix, G., Durrant, M. G., Ku, J., et al. (2025). Genome modeling and design across all domains of life with Evo 2. *bioRxiv* 2025.02.
- D'Halluin, A., Polgar, P., Kipkorir, T., Patel, Z., Cortes, T., & Arnvig, K. B. (2023). Premature termination of transcription is shaped by Rho and translated uORFs in *Mycobacterium tuberculosis*. *iScience*, 26(4), 106465.
- Dai, D., Deng, C., Zhao, C., et al. (2024). DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv:2401.06066*.
- Dalla-Torre, H., Gonzalez, L., Mendoza-Revilla, J., et al. (2025). Nucleotide Transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2), 287-297.
- Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv:2307.08691*.
- Di Salvo, M., Puccio, S., Peano, C., Lacour, S., & Alifano, P. (2019). RhoTermPredict: an algorithm for predicting Rho-dependent transcription terminators based on *Escherichia coli*, *Bacillus subtilis* and *Salmonella enterica* databases. *BMC Bioinformatics*, 20(1), 117.
- Ding, F., & Steinhardt, J. (2024). Protein language models are biased by unequal sequence sampling across the tree of life. *bioRxiv*.
- Gelsinger, D. R., Dallon, E., Reddy, R., Mohammad, F., Buskirk, A. R., & DiRuggiero, J. (2020). Ribosome profiling in archaea reveals leaderless translation, novel translational initiation sites, and ribosome pausing at single codon resolution. *Nucleic Acids Research*, 48(9), 5201-5216.
- Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.
- Hofmeyr, S., Egan, R., Georganas, E., et al. (2020). Terabase-scale metagenome coassembly with MetaHipMer. *Scientific Reports*, 10(1), 10689.
- Hong, J. (2026). Sparse upcycling for genomic foundation models: MoE architecture and expert specialization in GenomeOcean [Presentation]. Northwestern University.
- Hyatt, D., Chen, G.-L., LoCascio, P. F., Land, M. L., Larimer, F. W., & Hauser, L. J. (2010). Prodigal: prokaryotic gene recognition and translation initiation site identification. *BMC Bioinformatics*, 11, 119.
- Ji, Y., Zhou, Z., Liu, H., & Davuluri, R. V. (2021). DNABERT: pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. *Bioinformatics*, 37(15), 2112-2120.
- Jiang, A. Q., Sablayrolles, A., Roux, A., et al. (2024). Mixtral of Experts. *arXiv:2401.04088*.

- Lalanne, J.-B., Taggart, J. C., Guo, M. S., Herzel, L., Schieler, A., & Churchman, L. S. (2018). Evolutionary convergence of pathway-specific enzyme expression stoichiometry. *Cell*, 173(3), 749-761.
- Larralde, M. (2022). Pyrodigal: Python bindings and interface to Prodigal, an efficient method for gene prediction in prokaryotes. *Journal of Open Source Software*, 7(72), 4296.
- Li, Q., Wu, W., Zhu, Y., Feng, F., Ye, J., & Wang, Z. (2025). GenerAnno: A genomic foundation model for metagenomic annotation. *bioRxiv* 2025.06.
- Lin, Z., Akin, H., Rao, R., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130.
- Liu, O., Jaghouar, S., Hagemann, J., et al. (2025). METAGENE-1: Metagenomic foundation model for pandemic monitoring. *arXiv:2501.02045*.
- Lorenz, R., Bernhart, S. H., Höner zu Siederdissen, C., Tafer, H., Flamm, C., Stadler, P. F., & Hofacker, I. L. (2011). ViennaRNA Package 2.0. *Algorithms for Molecular Biology*, 6, 26.
- Matthews, B. W. (1975). Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta*, 405(2), 442-451.
- Meydan, S., Marks, J., Klepacki, D., Sharma, V., et al. (2019). Retapamulin-assisted ribosome profiling reveals the alternative bacterial proteome. *Molecular Cell*, 74(3), 481-493.
- Nakahigashi, K., Takai, Y., Shiwa, Y., Wada, M., Honma, M., Yoshikawa, H., & Tomita, M. (2016). Comprehensive identification of translation start sites by tetracycline-inhibited ribosome profiling. *DNA Research*, 23(3), 193-201.
- Nguyen, E., Poli, M., Faizi, M., et al. (2023). HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. *arXiv:2306.15794*.
- Nguyen, E., Poli, M., Durrant, M. G., et al. (2024). Sequence modeling and design from molecular to genome scale with Evo. *bioRxiv*.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Peters, J. M., Mooney, R. A., Grass, J. A., Jessen, E. D., Tran, F., & Landick, R. (2012). Rho and NusG suppress pervasive antisense transcription in *Escherichia coli*. *Genes & Development*, 26(23), 2621-2633.
- Petroni, E., Esnault, C., Tetreault, D., Dale, R. K., Storz, G., & Adams, P. P. (2023). Extensive diversity in RNA termination and regulation revealed by transcriptome mapping for the Lyme pathogen *Borrelia burgdorferi*. *Nature Communications*, 14, 3931.
- Sawyer, E. B., Cortes, T., et al. (2021). A snapshot of translation in *Mycobacterium tuberculosis* during exponential growth and nutrient starvation revealed by ribosome profiling. *Cell Reports*, 34(6), 108695.
- Schrader, J. M., Zhou, B., Li, G.-W., Lasker, K., et al. (2014). The coding and noncoding architecture of the *Caulobacter crescentus* genome. *PLOS Genetics*, 10(7), e1004463.
- Shine, J., & Dalgarno, L. (1975). Terminal-sequence analysis of bacterial ribosomal RNA. *European Journal of Biochemistry*, 57(1), 221-230.

- Solovyev, V., & Salamov, A. (2011). Automatic annotation of microbial genomes and metagenomic sequences. In R. W. Li (Ed.), *Metagenomics and its Applications in Agriculture, Biomedicine and Environmental Studies* (pp. 61-78). Nova Science Publishers.
- Steinegger, M., & Söding, J. (2017). MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35(11), 1026-1028.
- Su, W., Liu, M. L., Yang, Y. H., Wang, J. S., Li, S. H., Lv, H., et al. (2021). PPD: a manually curated database for experimentally verified prokaryotic promoters. *Journal of Molecular Biology*, 433(21), 166860.
- Wu, W., Zhou, Z., Riley, R., et al. (2025). Uncovering the genomic manifold via scalable learning from the global microbiome. *bioRxiv* 2025.01.30.635558.

Research Abstract

Locating promoters, ribosome-binding sites (RBS), and Rho-dependent terminators is foundational to prokaryotic genome annotation, yet the classical tools that do it are calibrated on a handful of model organisms and the dense genomic language models that might replace them treat every token with the same parameters. GenomeOcean-MoE, a sparsely upcycled 8-expert mixture-of-experts genomic foundation model, offers a third option: prior work showed its router encodes function in a genome-invariant way, distinct from the taxonomy-encoding hidden state. This work asks whether that routing channel identifies regulatory elements, and whether it does so better than classical annotation. I curated experimentally-derived ground truth for all three elements (25 organisms for promoters, 6 for RBS, 3 for Rho), validated every positive set against its own dinucleotide-preserving shuffle before modeling, and probed the frozen model with logistic regression under a six-class negative-control battery, reporting embedding, routing, and concatenated views separately. The central finding is a dissociation. For promoters and RBS the signal is carried almost entirely by the embedding and the router is effectively blind (routing MCC 0.017-0.246 for promoters, 0.040-0.359 for RBS, and 0.076 in the window-free per-token test), replicating across 8 bacterial phyla and 2 archaeal lineages. For Rho-dependent terminators the ordering inverts: routing exceeds embedding on the three controls that matter (D 0.498 vs 0.298, E 0.511 vs 0.233, F 0.968 vs 0.226), the only setting in this project where the MoE architecture contributes something a dense model could not. Scored head-to-head against five classical methods and two real published tools on identical held-out windows, GO-MoE ranks first of six on Rho, but second of seven on RBS behind real Prodigal (0.417 vs 0.470) and second of five on promoters behind a generic k-mer spectrum (0.265 vs 0.335). Three shuffle-controlled tests further show the Rho signal is compositional rather than arrangement-based, while two independent functional checks (Term-seq 3'-end enrichment, bicyclomycin readthrough) confirm the underlying windows are real. Expert routing is therefore a genuine but narrow instrument: it helps precisely where a positioned consensus motif does not exist, and adds nothing where one does.

Lesson Plan

K-12 Standards

1. **Subject Area of Interest:** Developing transformer models
2. **Grade Level:** High School Students
3. **Course Focus/Title:** AP Computer Science
4. **School Site/Setting/Type:** Urban Public & Private Schools

Standards:

Standard 1: CSTA 3B-AP-08: "Describe how artificial intelligence drives many software and physical systems."

- **Meaning:** Students learn what AI's place is in the world and effective ways to use it accordingly.
- **Example of The Standard Used In A Lesson:** Students can make a diagram of the flow of a transformer-based LLM, labeling how each layer contributes to generating text. They can then present how this same architecture powers real-world software like chatbots and search tools. They finish by discussing scenarios where wrapping, fine-tuning, or building a model from scratch would each be the smartest choice.

Standard 2: CSTA 3B-AP-09: "Implement an artificial intelligence algorithm to play a game against a human opponent or solve a problem."

- **Meaning:** Students learn the inner workings of AI models and learn how to adjust its structure for specific functions.
- **Example of The Standard Used In A Lesson:** Students can build a working chatbot in PyTorch, either by fine-tuning an existing LLM or coding a small transformer from scratch, so it can respond to user prompts. They then test and adjust their model's structure to improve how well it generates natural language or generate a specific language style of their choice.

Content Area:

I want to teach students how transformer-based models work and how to create one from scratch with tools like PyTorch, focusing the content on LLMs, which I believe to be the most understandable type of transformer model. I also want to go through every layer of complexity in creating chatbots: wrapping an existing LLM with existing instructions, fine-tuning LLMs, and creating them from scratch, and what situations we would want to use each of these methods for.

Rationale:

I chose CSTA 3B-AP-08 and 3B-AP-09 because they map almost one-to-one onto the two halves of my content area. My entire unit is centered on how AI systems, specifically LLMs, work under the hood, which directly addresses 3B-AP-08's focus on explaining AI's role in software. By having students build a transformer from scratch and dissect each layer, I move them well beyond a surface-level description toward a mechanistic understanding of how AI actually drives modern applications like chatbots. The second standard fit because teaching students to create a chatbot, whether by wrapping an existing LLM, fine-tuning, or building from scratch in PyTorch, is a hands-on implementation of an AI algorithm that solves a problem, namely natural language interaction. My progression through increasing layers of complexity also gives students multiple ways to satisfy the "implement" requirement of 3B-AP-09, which is why I felt these two standards paired so naturally.

I decided against using Common Core Math standards even though my content technically uses a lot of linear algebra and gradients, because I wanted my learning objectives to stay in the computer science realm rather than focusing on the math. This is honestly supposed to prove that you don't need to do most of the nitty-gritty math when you have LLMs! I also didn't find grade-band CSTA standards applicable, since the depth of building a transformer from scratch really belongs at the high school level, risky to go lower. I think these two standards match my content area pretty well, but I'm aware the material is a little ambitious, so I might need to reserve the "from scratch" portion for more advanced students.

K-12 Mapping

Two standards addressed in my lesson plan:

Standard 1: CSTA 3B-AP-08: "Describe how artificial intelligence drives many software and physical systems."

- **Meaning:** Students learn what AI's place is in the world and effective ways to use it accordingly.
- **Use In Lesson:** Students describe how a transformer LLM generates text one word at a time and diagram the prompt-to-response loop, then tie that same engine to tools they already use like chatbots and search autocomplete. The exit-ticket diagram is the check.

Standard 2: CSTA 3B-AP-09: "Implement an artificial intelligence algorithm to play a game against a human opponent or solve a problem."

- **Meaning:** Students learn the inner workings of AI models and learn how to adjust its structure for specific functions.
- **Example of The Standard Used In A Lesson:** Students implement a simple language model by hand, building a next-word table from a paragraph and generating sentences from it, then adjust it by adding more sample text, changing the seed word, or steering a

chatbot through prompting, and test how the output changes. The wrap vs. fine-tune vs. from-scratch task pushes the same idea into a design decision.

What content do students need to know before they can engage with the new ideas in your lesson?

- Students need to have used a chatbot or texting autocomplete, be able to read a short paragraph and pick out individual words, and understand that a program follows a set of rules. From AP CS they should already have loops and simple algorithms, since building a next-word table and looping through it to generate a sentence depends on both.

Regarding the standards you've selected for your lesson, briefly summarize how both younger and older students are expected to engage with these standards?

- Both standards are 3B (grades 11-12), the most advanced band, so younger students never meet them head-on. In the lower bands they engage with AI-adjacent ideas instead: modeling everyday processes and decomposing steps into algorithms, discussing AI's role and impact in society, and translating real-world phenomena into data and simple models. Older students at 3B do the full standard, describing how AI actually drives real systems and implementing and adjusting an AI algorithm themselves, which is exactly what my lesson targets.

Final 5E Lesson Plan

5E Lesson Plan

Cal Teach Student Name(s): Gilberto Damian

Mentor Teacher Name: Dr. Morgan Mitchell (EDSTEM 189 summer seminar)

School/Room #: Hypothetical urban public/private high school

Grade Level and Subject: High School (grades 11-12), AP Computer Science

Date/Time to be Taught: Summer 2026 (10-minute Engage/Explore segment taught to EDSTEM 189 peers)

Lesson Source(s): Code.org. (2023). How AI Works, Unit 1, Lesson 4: Chatbots and Large Language Models. <https://studio.code.org/courses/how-ai-works-2023/units/1/lessons/4> Found through the National AI Literacy Day resource library (ailiteracyday.org). I use its “what is an LLM” and next-word-prediction ideas for the Engage and Explain phases, and I added an unplugged “be the language model” activity and a wrap/fine-tune/from-scratch discussion for the Explore and Elaborate phases.

Focus/Essential Question: If a chatbot is really just predicting the next word, how does it end up sounding human, and how could we change the way it writes?

Student Learning Objectives: By the end of the lesson, students will be able to:

- (CSTA 3B-AP-08) Describe how a transformer-based large language model generates text by predicting one word at a time, and draw and label a simple diagram of the flow from a prompt to the finished text, then explain how that same kind of model powers everyday tools like chatbots and search.

- (CSTA 3B-AP-09) Implement and adjust a simple language model, either by building a small next-word predictor by hand or by steering an existing chatbot through prompting, and test how their changes affect the text it produces.
- (Stretch) Argue when it makes the most sense to wrap an existing model, fine-tune one, or build one from scratch, using a chatbot they care about as the example.

Common Core or NGSS Standards:

- CSTA 3B-AP-08 (Grades 11-12, Algorithms & Programming): “Describe how artificial intelligence drives many software and physical systems.”
- CSTA 3B-AP-09 (Grades 11-12, Algorithms & Programming): “Implement an artificial intelligence algorithm to play a game against a human opponent or solve a problem.”

(These are Computer Science Teachers Association standards; the assignment allows CS standards in place of Common Core or NGSS.)

Student Prior Knowledge:

- Expected: students have used chatbots or texting autocomplete, can read a short paragraph and identify words, and understand that a program follows rules. In AP CS they have already seen loops and simple algorithms.
- Possible gaps or misconceptions: that AI “understands” or “looks things up,” that it is always correct, that it copies whole sentences instead of generating word by word, and confusion between using an AI tool and building one.

Lesson Agenda For My Students:

- Guess the next word (warm up)
- Be the language model: build a next-word table and generate a sentence
- Watch: how do real chatbots actually work?
- Three ways to build a chatbot: wrap, fine-tune, or from scratch
- Exit ticket: draw how an LLM writes

Lesson Rationale:

These activities move students from a tool they already use to the mechanism behind it. The unplugged next-word activity makes the core idea of an LLM concrete and needs no computers, which keeps it accessible. Building and then adjusting a simple model, and choosing wrap versus fine-tune versus from scratch, gives students a hands-on implementation and a real design decision, which is what the practice standard asks for. The diagram exit ticket lets every student demonstrate the describe-the-system objective from the content standard.

Materials and Technology List:

- One projector and one chatbot open in a browser (teacher station only).
- Printed sample paragraph, one per group of 3 to 4 (about 8 to 10 copies for a class of 32); a few sentences I wrote myself so there is no copyright issue.
- Next-word table worksheet, one per student (about 32), plus spares.
- Index cards or scratch paper, one small stack per group.
- Exit-ticket half-sheets, one per student (about 32).
- Optional: a few laptops or tablets for the plugged style-tuning extension.

Packaging: I pre-collate one folder per group with the paragraph, worksheets, and index cards so distribution takes under a minute.

Preparation Tasks:

- Write and print the sample paragraph and worksheets, and collate the group folders.
- Load the chatbot and the Code.org video, and test the projector and sound.
- Prepare the vocabulary anchor chart and the board agenda.

- Pre-write two scenario cards per group for Elaborate.
- Make the exit-ticket rubric.

Safety Concerns:

Physical risk is low since the lesson is mostly paper-based. The main concern is privacy: I set a rule that students do not type personal information into the chatbot, I keep the class on one teacher-controlled chatbot for the demo, and if the optional laptop extension is used I supervise prompts and remind students that chatbot output can be wrong or inappropriate and should always be checked.

Lesson Title: How Large Language Models Work: Predicting the Next Word and Building a Simple Chatbot

Engage: <i>Activities that engage students' interest and build connections to their lives and prior knowledge.</i>	<i>Previous Experience and Baseline Learning</i>
Introduce and engage: I put a chatbot on the projector and type half a sentence, like “The best part of summer break is...” Before I press enter, I ask the class to call out the word they think comes next and I take three or four guesses out loud. Then I let the model finish and we compare. I say the key line: the model is not looking anything up and it is not thinking, it is just guessing the next word, over and over, very fast. Connect to their lives / elicit prior knowledge: Where have you seen this before? (texting autocomplete, search, ChatGPT.) Has autocomplete ever guessed the exact word you meant? Has it ever been confidently wrong? I expect students to name texting and ChatGPT quickly and to disagree about how “smart” it is. Transition to Explore: “If a chatbot is really just picking likely next words, we can do that too, by hand. Let’s become the model.” Academic language / ELL supports: I post key terms (predict, next word, model, prompt) with a one-line definition and an icon on an anchor chart, I use a turn-and-talk before whole-class share so students rehearse with a partner first, and guesses can be spoken or written. Time: 10 minutes.	<u>Watch for:</u> Engagement shows up as students blurting guesses, laughing at wrong completions, and connecting it to apps they use. Prior knowledge will range from students who use ChatGPT daily to students who have only seen texting autocomplete. <u>Response:</u> If the room is quiet, I revisit prior knowledge by pulling up phone-style autocomplete first, which everyone recognizes, before showing the chatbot. I keep the seed sentence low-stakes and funny so guessing feels safe.
Explore: <i>Hands-on tasks designed to explore ideas and to develop skills together.</i>	<i>Focus, Involvement, Collaboration, Results, and Recording</i>
Directions and demonstration: I model one round on the board. I take a short sample paragraph and show how to build a “next word” table: pick a word, then list the words that come right after it in the text. Then, starting from a seed word, I generate a new sentence by repeatedly choosing a likely next word from the table. I do two sentences so the process is clear. Grouping and materials: students work in groups of 3 to 4. Each group gets one printed paragraph and a stack of cards or a shared worksheet. Roles rotate: a reader, a recorder, and a “generator” who picks the next word. Handouts (attached): the sample paragraph and a blank next-word table worksheet. During the activity: I circulate and ask groups to read me their generated sentence and say why it sounds “almost real but a little off.” I nudge them to notice that more sample text gives more natural output. Groups that finish early and have a computer open a chatbot and try to make it write in a specific style using only the prompt. Clean-up and transition: “Finish the sentence you’re on, circle your favorite generated line, and get ready to share what you noticed about how your table decided what came next.”	<u>Watch for:</u> I want to see students building tables from evidence in the text, debating which next word is “allowed,” and realizing the output is plausible but not meaningful. Constructing meaning looks like a group saying “it only knows what usually comes next, not what it means.” <u>Response:</u> Prompts: Have you noticed which words have lots of options after them? What happens if your sample text is longer? How would you describe why the sentence sounds almost right? Can you find a way to make it sound more formal or more casual?

Academic language / ELL supports: the worksheet has a sentence frame (“Our model chose ____ because ____”), key terms stay on the anchor chart, and groups talk before they write. Time: 25 minutes.	
Explain: <i>Students explain the phenomena they explored and discuss their different ideas and perspectives.</i>	<i>Participation, Reporting, Debating, and Evidence-Based Reasoning</i>
Organizing the discussion: each group reports one generated sentence and one thing they noticed, and I chart the noticings on the board. Then we watch the Code.org “How AI Works: Chatbots and Large Language Models” video (about 4 minutes). Afterward I draw the real loop and map it onto what they just did: a prompt goes in, the model predicts the next word, that word is added, and it repeats until the response is done. Prompts: What did you notice about how your table picked words? How is a real LLM different from your table? (Toward: it learned from a huge amount of text, and a transformer lets it weigh the whole sentence, not just the last word.) What evidence from your activity supports the idea that it is “just prediction”? Big conclusions: an LLM generates text one word at a time from patterns it learned, the same engine powers chatbots and search autocomplete, and because it only predicts likely words it can sound confident and still be wrong. Summary: I restate the loop in one sentence and post the vocabulary: token/word, prompt, training data, model, generate, transformer. Academic language / ELL supports: sentence frames for reporting (“We noticed ____, which shows ____”), the vocabulary chart, and I revoice partial answers into full academic phrasing. Time: 20 minutes.	<u>Watch for:</u> Common misconceptions: that the model “understands” or “looks up” answers, that it is always right, or that it copies sentences whole rather than building them word by word. <u>Response:</u> I prompt the group to reconcile a misconception with their own activity (“your table didn’t understand anything, so what was it doing?”). For reluctance to share I use think-pair-share and cold-call from volunteers’ partners; for over-sharing I use a “one idea then pass” rule; for a range of understanding I give a diagram to extend to strong students and a fill-in version to those who need scaffolding, so everyone reaches the predict-the-next-word objective.
Elaborate: <i>Teacher-stimulated application and clarification of concepts, skills, attitudes, processes or terminology.</i>	<i>Demonstrated Understanding, Use of Skills, and Other Applications</i>
Apply to a new situation: I introduce three ways to get a chatbot that behaves how you want. Wrapping means giving an existing model good instructions in the prompt. Fine-tuning means training an existing model a little more on examples of the style you want. Building from scratch means training your own model from the ground up. Groups get a scenario (a homework helper, or a bot that always answers like a pirate) and must pick one approach and defend the trade-offs of cost, data, time, and control. Connect to prior understanding: I tie each option back to the activity. Wrapping is like changing your seed word and prompt, fine-tuning is like giving your model a bigger and more specific sample to learn from, and from-scratch is building the whole next-word table yourself. Vocabulary review: wrap = instruct an existing model; fine-tune = keep training an existing model on your examples; from scratch = build and train your own. If	<u>Watch for:</u> Whether students can match a scenario to a sensible approach and justify it, not just pick one. A student who needs help will choose “from scratch” for everything without weighing cost. <u>Response:</u> I support strugglers with a two-column pros/cons starter and a guiding question (“do you really need a new model, or just a good instruction?”), and each group defends against one

computers are available, groups try the wrap option live by prompting a chatbot into a style. Academic language / ELL supports: a decision chart with the three options and a sentence frame (“We chose to ____ because it gives us ____ but costs us ____”). Time: 20 minutes.	challenge from another group so the reasoning gets tested.
Evaluate: <i>Observe and adjust your lesson as you teach.</i>	<i>Assessment and Evidence of Learning</i>
Assessment (exit ticket): (1) Draw and label the flow of an LLM generating text, from prompt to finished response (assesses CSTA 3B-AP-08 and objective 1). (2) In one sentence, explain what “predicting the next word” means and why it can make an LLM sound sure of itself and still be wrong. (3) Pick wrapping, fine-tuning, or building from scratch for a given scenario and justify the choice (supports CSTA 3B-AP-09 and objective 3). Formative evidence: I also collect the Explore next-word tables and use the cold-calls during Explain. Adjusting as I teach: if many exit tickets miss the “one word at a time” idea, I reteach the loop with a quick whole-class round before moving on. Scoring: a three-level rubric on the diagram and the design justification (cannot yet describe the loop / can describe but not justify / can do both clearly). Time: 15 minutes.	<u>Watch for:</u> Whether the diagram shows the repeating predict-add-repeat loop, and whether the justification names a real trade-off. <u>Response:</u> For students who struggle, I accept a labeled verbal explanation instead of a drawing, and I conference briefly with anyone whose diagram is missing the loop so they leave with the core objective met.

TLDR; Breakdown

Engage (10 minutes)

- Warm-up: The teacher projects a chatbot, types half a sentence (like "The best part of summer break is..."), and has the class guess the next word
- Discussion: The class compares their guesses to the chatbot's output and discusses their prior experiences with autocomplete tools, search, and ChatGPT

Explore (25 minutes)

- Demonstration: The teacher models how to build a "next word" table using a short paragraph on the board
- Group Activity: Students break into groups of 3 to 4. Using a printed paragraph, they build their own next-word tables and generate a new sentence by picking likely next words
- Wrap-up & Extension: Groups read their generated sentences aloud
- Early finishers can use a computer to try making a real chatbot write in a specific style

Explain (20 minutes)

- Reporting: Groups report one generated sentence and one thing they noticed during the activity
- Video: The class watches a 4-minute Code.org video titled “How AI Works: Chatbots and Large Language Models”

- Discussion & Diagramming: The teacher draws the real loop of an LLM on the board (prompt in, model predicts next word, word is added, loop repeats) and connects it to the table activity they just did

Elaborate (20 minutes)

- Concept Introduction: The teacher introduces three methods for making a chatbot behave a certain way: wrapping, fine-tuning, and building from scratch
- Group Activity: Groups are given a scenario (such as building a homework helper or a pirate bot) and must choose one of the three approaches, defending their choice based on trade-offs like cost, data, time, and control
- Optional: If computers are available, students can practice the "wrap" option live

Evaluate (15 minutes)

- Exit Ticket: Students complete an exit ticket where they must:
 - Draw and label the flow of an LLM generating text
 - Explain what “predicting the next word” means in one sentence
 - Choose and justify an approach (wrapping, fine-tuning, or from scratch) for a specific scenario

Reflections

Reflection #1

AI is already integrated into how research gets done. In my own work, AI isn't just what I study, it's literally the instrument, learning statistical patterns across microbial genomes to flag features that would take classical tools much longer to find. The readings state that computing has moved from capturing data to detecting patterns and automating decisions about what those patterns mean. The advantages are speed, scale, and the ability to model messy variability or generate hypotheses no one would have time to enumerate. But the problem is that these models are approximations that are narrower than the reality they stand in for. In genomics, that might mean that a model annotates good on familiar organisms and quietly fails on divergent ones. And if I trust it because the output "looks good", I've introduced bias that I can't make real conclusion from.

When we let AI interpret and act on data, we're delegating judgment, and both the UNESCO guidance and the Dept of Ed report talk about this same point. We have to keep a human in the process, AI should never set the goals. People should stay responsible for deciding what the patterns mean. The failure in using AI isn't it going rogue, it's humans over-trusting a tool and under-applying their own judgment. Making plausible-looking results built on bad data that nobody catches. Whether AI makes research better, or just faster at being wrong, depends on choices that people like me should make, not the technology itself.

Reflection #2

To find the amount of pH change needed for humans to tell when there is a significant change in acidity for sour candies, there's a lot that we have to establish. Firstly, we don't know if there's a critical point in which the pH of a candy is too high to tell the difference in acidity. We would need to start the test with candies that don't have any sourness in them (neutral pH) and then give them candies of decreasing pH one at a time. To cleanse the palate after each candy would actually be detrimental, because it would increase the chance of forgetting what the previous sourness experience felt like. An immediate recognition of an increase in sourness should be plenty proof that the pH increase was noticed. For each interval, we would record each person's answer (100-person sample) on whether the sourness increased, decreased, or stayed the same, then find where most responses clustered at certain points of pH. Everyone's sour tolerance will be different, which adds difficulty to this methodology, but nonetheless seems like the best way to get the average pH intervals that people perceive in sour candies. It's possible that there are very little perceived differences in sourness beyond a certain point in pH, but I suspect that in the beginning stages of increasing the pH, there will be significant differences in sourness perception.

Prompt: You are working at a food company to develop a new sour candy. You know that perceived sourness depends on acidity, but you do not know how big of a change in pH is necessary for humans to tell the difference. A coworker claims that humans can detect

differences of 1.5 pH units or more. Write a one-paragraph (6 sentences maximum) experimental design to test this claim. Be very detailed in the data used, sample size, data recording methods, etc.

Response: Prepare six identical sour candy formulations that differ only in pH (for example, pH 2.0, 2.5, 3.0, 3.5, 4.0, and 4.5), verifying each batch with a calibrated pH meter while keeping flavor, color, texture, sugar concentration, temperature, and size constant. Recruit 100 adult participants with no known taste disorders, and use a randomized, double blind paired comparison design in which each participant tastes candy pairs with pH differences of 0.5, 1.0, 1.5, and 2.0 units in random order, rinsing with water and waiting one minute between samples to minimize carryover effects. For every pair, participants record on an electronic survey whether the two candies taste different in sourness (yes/no), which candy is more sour, and their confidence on a 5 point scale. Record each participant's responses, the pH values of the pair presented, trial order, age, and sex in a spreadsheet for analysis. Calculate the proportion of correct identifications for each pH difference and use a chi square test (or logistic regression) to determine whether detection rates at a 1.5 pH difference are significantly higher than chance and whether they differ from smaller pH differences. If participants reliably identify sourness differences at 1.5 pH units but not at smaller differences, the coworker's claim is supported; otherwise, it is rejected.

One thing that my design does better is recognizing that cleansing your palate can negatively affect the experiment. However, ChatGPT might find this to be standard since experiments such as this that involve smell and taste often do this with something like coffee beans. Albeit ChatGPT could be right, and I could be wrong. However, my response is still better in the sense that my personal experience and unique perspective can yield more results than what ChatGPT's standard methodology provides. One thing that ChatGPT did better in its design is adding excruciating detail about pH values, timing, methods of recording opinions, etc. Having this amount of detail in research methodology is crucial to not doing guesswork later. I generalized a lot in my research design, as to give a simple pitch as to what we could roughly do, ChatGPT went all the way, granted I told it to do that in the prompt.

Reflection #3

For my research project this summer, I've done a lot of data analysis around refining different negative controls for the same experiment and comparing the Matthews Correlation Coefficient (MCC) values between those results. I also had to normalize datasets collected from several different sources so that they were all in the same format before I could build consistent training and validation windows for my binary classifiers. I did this because I wanted to test how well GenomeOcean-MoE could recognize promoters in DNA when each classifier was trained against a different definition of what "not a promoter" meant. This is important because a model that only performs good against one negative control may not actually generalize to other situations. By comparing the results across multiple classifiers, I can better determine whether the model is actually learning the biological patterns associated with promoters instead of memorizing a specific dataset.

For how I can bring my methods of data analysis into my future teaching, one of the biggest lessons I've learned is that good data alone doesn't guarantee good conclusions. My mentor has emphasized that even if a dataset comes from a reliable source, an experiment with flawed assumptions or poorly designed controls can still lead to misleading results. I think that this is something that is easy to overlook because it's tempting to trust the output of an experiment without questioning how the data was prepared in the first place. I would encourage students to spend just as much time evaluating their data and experimental design as they do interpreting their results. Having good data is important, but knowing how to use that data correctly is what makes an experiment valid and actually mean something.

Reflection #4

This question is interesting, and I want to start by summarizing my answer to this question, and then going in depth: Science is done as a result of social and cultural values, but is carried out in a way that transcends national and cultural boundaries. In science, there are infinite questions and approaches to uncover the answers of the world that would be completely valid and give us results based on our experiments. However, we usually don't put science into the ether. We do science to solve problems, and problems don't exist without subjectivity.

Nothing that we believe is objective, that's why it's called a belief. Even some cultures have beliefs on things most might find objective, like who should live or not. We use science to answer questions that we think SHOULD be answered. For example, we do research on what causes Parkinson's disease and how we should cure it. Somebody could claim that it doesn't matter if people with Parkinson's suffer, it's a small minority so why should we waste resources on it? But a lot of humans share the culture of universal kindness and assistance, "all for one, one for all". So we start a science to find answers to what we think will support our culture. Frontal lobotomies were invented in the 30s to cure mental illnesses, or "tame women"! A crazy example, but nonetheless an example of how social and cultural values drove science.

However, if science is done in an absolutely correct manner; no biases, no confounders, true experiences and findings presented in complete transparency manner, down to the definition, then regardless of the motive, the results should speak for themselves, and sometimes give us universal answers that conflict with the general population's social and cultural beliefs. For example, Galileo was tried by the Roman Catholic Inquisition for advocating heliocentrism as physical truth. He was found "vehemently suspected of heresy," forced to publicly retract his views, and sentenced to house arrest for the rest of his life. The scientific method is not fundamentally affected by social and cultural biases, it's only contaminated and ruined with it. However, the questions we've asked based on our biases are what made science exist in the first place.

Research Talk Reflection

My poster presentation ended up looking a little different than how it would normally run, since I presented in front of the class instead of alongside everyone else at once, but the experience itself was still a good one. I was able to get good questions from people about my work, and I was thrilled to have the chance to actually show people what I'd been working on. Explaining my project out loud made me realize how much I actually understood it, and I was able to walk through the main points clearly and thoroughly. Regardless of the different format, everything turned out pretty good overall.

However, if I'm being honest with myself, I think I could've done better in preparing my talking points, specifically around my results. I found myself spending too much time explaining general background concepts that I could've covered much faster, which ate into the time I had to really dig into what I found. To improve next time, I want to shorten those general explanations so I can get to the results faster and spend more of my talking points making sure people walk away actually understanding what I discovered.

Overall Reflection

This class taught me a lot overall, and even though my schedule was modified along the way, I was still able to get a lot out of it. Talking with my peers who were working in completely different labs and settings than me helped me build critical thinking skills that I think are going to be essential for all of us moving into this new AI driven world. I also really liked the talks we had, since I got to see how people working in real sciences and education are actually using AI to drive innovation forward and find connections between things we hadn't seen before. It was good to have the chance to challenge them on their work too, in hopes that it would push their final results to be even stronger. Getting to teach my own lesson was great as well. I got a lot of good feedback on it and it seemed like people generally had a lot of fun with it.

Dr. M is a great instructor, with a ton of enthusiasm, energy, knowledge, and fun packed into every class. Dr. M and Ari were both super understanding of whatever support I needed along the way, and I was genuinely happy to be in class every Wednesday. Looking back, I think this class prepared me well for teaching, and I'm excited to actually put these skills to use in the coming years.